

Assignment-3

1. Valid Parentheses:

The screenshot shows the LeetCode interface for the 'Valid Parentheses' problem. On the left, a bar chart displays the performance of various submissions, with the selected solution being the fastest. The main area shows the Java code for the solution, which uses a stack to validate the parentheses. The code is as follows:

```
1 class Solution {
2     public boolean isValid(String s) {
3         Stack<Character> st = new Stack<>();
4         if(s.length() == 1) return false;
5         for(int i=0; i<s.length(); i++){
6             if(s.charAt(i) == '(' || s.charAt(i) == '[' || s.charAt(i) == '{') {
7                 st.push(s.charAt(i));
8             } else {
9                 if(st.isEmpty()) return false;
10                Character ch = st.peek();
11                if(
12                    s.charAt(i) == ')' && ch == '(' ||
13                    s.charAt(i) == ']' && ch == '[' ||
14                    s.charAt(i) == '}' && ch == '{'
15                ){
16                    st.pop();
17                } else {
18                    return false;
19                }
20            }
21        }
22        return st.isEmpty();
23    }
24 }
```

Below the code, there are links for 'Testcase' and 'Test Result'. At the bottom, there are suggestions for other problems: '32. Longest Valid Parentheses' and '301. Remove Invalid Parentheses'.

2. Remove All Adjacent Duplicates In String:

The screenshot shows the LeetCode interface for the 'Remove All Adjacent Duplicates In String' problem. The problem description on the left explains that adjacent duplicate letters should be removed until no more can be removed. Example 1 shows 'abbaca' becoming 'ca', and Example 2 shows 'azxxzy' becoming 'ay'. The main area shows the Java code for the solution, which uses a stack to remove adjacent duplicates. The code is as follows:

```
1 class Solution {
2     public String removeDuplicates(String s) {
3         Stack<Character> st = new Stack<>();
4         StringBuilder sb = new StringBuilder();
5         for(int i=0; i<s.length(); i++){
6             if(st.isEmpty()){
7                 Character ch = st.peek();
8                 if(ch == s.charAt(i)) {
9                     st.pop();
10                    continue;
11                }
12            }
13            st.push(s.charAt(i));
14        }
15    }
16 }
```

Below the code, there are links for 'Testcase', 'Test Result', and 'Accepted'. The submission statistics show that the solution is 'Accepted' and has passed 106 / 106 testcases. The runtime is 39 ms, which beats 18.09% of submissions, and the memory usage is 47.34 MB, which beats 28.59% of submissions. The submission was made by 'Raj Vardhan Singh Gangwar' on May 05, 2026 at 17:26.

3. Palindrome Linked List

234. Palindrome Linked List Solved

Given the `head` of a singly linked list, return `true` if it is a *palindrome* or `false` otherwise.

Example 1:

Diagram: 1 → 2 → 2 → 1

Input: `head = [1,2,2,1]`
Output: `true`

Example 2:

Diagram: 1 → 2

Input: `head = [1,2]`
Output: `false`

Constraints:

- 1 ≤ `head.val` ≤ 10⁴
- 1 ≤ `head.length` ≤ 10⁵

Code:

```
Java
17     st.push(trav.val);
18     trav = trav.next;
19 }
20
21     ListNode travRev = head;
22     while(travRev != null){
23         if(st.peek() != travRev.val) return false;
24         st.pop();
25         travRev = travRev.next;
26     }
27     return true;
28
29
30 }
```

Testcase: Accepted 93 / 93 testcases passed

Raj Vardhan Singh Gangwar submitted at May 05, 2026 17:38

Runtime: 18 ms | Beats 5.91%

Memory: 101.62 MB | Beats 8.76%

4. Middle of the linked list

876. Middle of the Linked List Solved

Given the `head` of a singly linked list, return the *middle node* of the linked list.

If there are two middle nodes, return the *second middle node*.

Example 1:

Diagram: 1 → 2 → 3 → 4 → 5 (Node 3 is highlighted)

Input: `head = [1,2,3,4,5]`
Output: `[3,4,5]`
Explanation: The middle node of the list is node 3.

Example 2:

Diagram: 1 → 2 → 3 → 4 → 5 → 6 (Node 4 is highlighted)

Input: `head = [1,2,3,4,5,6]`
Output: `[4,5,6]`
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Code:

```
Java
14     ListNode slow = head;
15     ListNode fast = head;
16
17     while(fast != null && fast.next != null){
18         slow = slow.next;
19         fast = fast.next.next;
20     }
21     return slow;
22
23
24 }
```

Testcase: Accepted 36 / 36 testcases passed

Raj Vardhan Singh Gangwar submitted at May 05, 2026 17:50

Runtime: 0 ms | Beats 100.00%

Memory: 42.93 MB | Beats 34.35%

5. Delete the Middle node of a given Linked list

The screenshot shows the LeetCode interface for the problem "Delete the Middle Node of a Linked List". The problem description states: "You are given the head of a linked list. Delete the middle node, and return the head of the modified linked list. The middle node of a linked list of size n is the $\lfloor n / 2 \rfloor$ node from the start using 0-based indexing, where $\lfloor x \rfloor$ denotes the largest integer less than or equal to x. For n = 1, 2, 3, 4, and 5, the middle nodes are 0, 1, 1, 2, and 2, respectively."

Example 1: A linked list with nodes 1, 3, 4, 7, 1, 2, 6. The middle node (7) is highlighted in red. Input: head = [1,3,4,7,1,2,6]. Output: [1,3,4,1,2,6].

Example 2: A linked list with nodes 1, 2, 3, 4, 5. The middle node (3) is highlighted in red. Input: head = [1,2,3,4,5]. Output: [1,2,4,5].

The Java solution is as follows:

```
public class Solution {
    public ListNode deleteMiddle(ListNode head) {
        if (head == null) return head;
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

Test results: Accepted 70 / 70 testcases passed. Runtime: 5 ms, Beats 3.07%. Memory: 202.72 MB, Beats 22.92%.

6. Linked List Cycle

The screenshot shows the LeetCode interface for the problem "Linked List Cycle". The problem description states: "Given head, the head of a linked list, determine if the linked list has a cycle in it. There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false."

Example 1: A linked list with nodes 3, 2, 0, -4. The tail node (-4) points back to the node at index 0 (value 3). Input: head = [3,2,0,-4], pos = 1. Output: true.

Example 2: A linked list with nodes 1, 2, 3, 4, 5. The tail node (5) points to null. Input: head = [1,2,3,4,5], pos = -1. Output: false.

The Java solution is as follows:

```
public class Solution {
    public boolean hasCycle(ListNode head) {
        if (head == null || head.next == null) return false;
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow == fast;
    }
}
```

Test results: Accepted 29 / 29 testcases passed. Runtime: 0 ms, Beats 100.00%. Memory: 46.93 MB, Beats 15.95%.